

LEVEL 1 (Q1-Q10)

Promise Fundamentals

Q1

Create your own Promise that resolves after 2 seconds with:

Hello World

Q2

Create a Promise that rejects after 3 seconds with:

Server Down

Q3

Create a Promise that randomly resolves or rejects.

Q4

Create a Promise that checks age.

age >= 18

resolve

otherwise reject.

Q5

Create a Promise that validates password length.

Q6

Create a Promise that validates email.

Q7

Create a Promise that simulates bank withdrawal.

Q8

Create a Promise that simulates ATM transaction.

Q9

Create a Promise that resolves with user object.

Q10

Create a Promise that rejects with custom error object.

LEVEL 2 (Q11-Q20)

Then Chaining

Q11

Start with:

`Promise.resolve(10)`

Convert:

10



20



40



80

Using chaining.

Q12

Create a chain that converts:

Ram



RAM



RAM MOHAN



RAM MOHAN DIXIT

Q13

Create a chain that processes product price.

1000



Add GST



Apply Discount



Final Price

Q14

Create a chain for:

Login



Get User

↓
Get Orders
↓
Get Payment

Q15

Create a chain for movie booking flow.

Q16

Create a chain for food ordering flow.

Q17

Create a chain for college admission flow.

Q18

Create a chain for train ticket booking flow.

Q19

Create a chain where each step waits 1 second.

Q20

Create a chain of 10 `.then()` calls.

LEVEL 3 (Q21-Q25)

Catch Propagation

Q21

Throw error inside first `.then()`.

Handle in catch.

Q22

Throw error inside third `.then()`.

Observe skipped thens.

Q23

Create chain:

Step1

↓

Step2

↓

Error

↓

Catch

Q24

Recover from error inside catch and continue chain.

Q25

Create multiple catches and observe which one executes.

LEVEL 4 (Q26-Q30)

Callback → Promise Conversion

Q26

Convert callback based greeting function into Promise.

Q27

Convert callback based calculator into Promise.

Q28

Convert callback based login into Promise.

Q29

Convert callback based file download simulation into Promise.

Q30

Convert callback based weather API simulation into Promise.

LEVEL 5 (Q31-Q35)

Promise.all()

Q31

Fetch:

User
Orders
Products

using Promise.all.

Q32

Create 5 promises with different delays.

Run using Promise.all.

Q33

Make one Promise reject.

Observe Promise.all behavior.

Q34

Create dashboard loader using Promise.all.

Q35

Build parallel file downloader using Promise.all.

LEVEL 6 (Q36-Q38)

Promise.allSettled()

Q36

Create:

3 Success

2 Fail

Return all results.

Q37

Create admin dashboard loader.

Some APIs fail.

Still show remaining data.

Q38

Build batch upload simulator using `Promise.allSettled`.

LEVEL 7 (Q39)

`Promise.race()` & `Promise.any()`

Q39-A

Create:

Server1 → 5 sec

Server2 → 2 sec

Server3 → 1 sec

Return fastest response using race.

Q39-B

Create:

Server1 → Reject

Server2 → Reject

Server3 → Resolve

Return first success using any.

LEVEL 8 (Q40) FINAL BOSS

Build a Complete Promise-Based Food Delivery System

Flow:

Login



Fetch User



Fetch Restaurants



Fetch Menu



Place Order



Process Payment



Generate Invoice



Send Email



Track Order



Deliver Order

Requirements:

- ✓ Every step returns Promise
 - ✓ Use chaining
 - ✓ Handle failures
 - ✓ Use catch propagation
 - ✓ Use finally
 - ✓ Simulate delays
 - ✓ At least 3 random failure points
-



INTERVIEW OUTPUT ROUND (BONUS)

IQ1

```
Promise.resolve("A")  
  .then(console.log);
```

```
console.log("B");
```

IQ2

```
console.log("A");
```

```
Promise.resolve()  
  .then(()=>{  
    console.log("B");  
  });
```

```
console.log("C");
```

IQ3

```
Promise.resolve(10)  
  .then(num=>num*2)  
  .then(num=>num+5)  
  .then(console.log);
```

IQ4

```
Promise.reject("Error")  
.catch(console.log);
```

IQ5

```
Promise.resolve()  
.then(()=>{  
  throw new Error("Boom");  
})  
.catch(err=>console.log(err.message));
```

IQ6

```
Promise.resolve()  
.then(()=>{  
  return Promise.resolve(100);  
})  
.then(console.log);
```

IQ7

```
Promise.resolve()  
.then(()=>{  
  console.log("A");  
})  
.then(()=>{  
  console.log("B");  
});
```

IQ8

```
Promise.all([  
  Promise.resolve(1),  
  Promise.resolve(2),  
  Promise.resolve(3)  
)  
.then(console.log);
```

IQ9

```
Promise.all([
  Promise.resolve(1),
  Promise.reject("Failed"),
  Promise.resolve(3)
])
.catch(console.log);
```

IQ10

Predict complete output:

```
console.log("Start");
```

```
Promise.resolve()
  .then(()=>{
    console.log("A");
    return Promise.resolve("B");
  })
  .then(console.log);
```

```
console.log("End");
```